jaegeon.dev

DEVELOPER PORTFOLIO

VOL. 01 / 2026

복잡한 업무를 풀고,
데이터 정합성을 지킵니다.

백엔드 개발자 유재건 · 엠로(emro), 2024.09 — 현재

Java·Spring 기반 B2B 서비스에서 업무 로직, 데이터 구조와 운영 안정성을 개선합니다.

업무 이해 · 정합성

원가 계산의 차이를 추적

최종 단가 0.07% 차이에서 환차액 누락 원인을 분석했습니다.

운영 안정성

처리 단계와 트랜잭션 분리

처리중 상태와 건별 신규 트랜잭션을 적용하고 오류 로그를 별도로 보존했습니다.

대량 데이터 처리

약 2억 건 정리 대상 분석

ROWID 기반 분할 삭제와 주기적 COMMIT으로 운영 부하를 고려했습니다.

01 / EXPERIENCE

제품을 만들고,
운영하며 배운 것

업무를 데이터 구조로 옮기고,
운영 중인 서비스의 안정성을 높였습니다.

엠로 emro

재직 중

개발 사원 · 정규직
2024.09 — 현재

- 현대모비스 원가재료비 관리 프로젝트의 원가 산정 로직 개발
- emroCloud 고객문의 데이터 구조 개선 및 Jira 연동 API 개발
- 일괄결재 동시성 제어, 오류 로그 보존과 대량 운영 데이터 정리

Java | Spring Framework | Oracle / DB2 | MyBatis

↳ 분석과 공유도 개발의 일부로 생각합니다.
Claude Code를 활용해 SaaS 구매 서비스를 분석하고, 온보딩 문서를 작성·공유했습니다.

---

02 / PROFESSIONAL WORK

실무에서 해결한 문제들

정합성 · 안정성 · 데이터 처리

현업 요구사항을 계산 로직과 데이터 구조로 옮기고, 운영 중 드러난 문제를 처리 방식의 개선으로 연결했습니다.

PRODUCTION EXPERIENCE / 01

0.07%

최종 단가 차이의 원인 분석

원가 로직 개발 2026.05 — 2026.09

원가 산정의 정합성 확보

현대모비스 원가재료비 관리

Java DB2 MyBatis Rhino JavaScript

> **계산 결과의 작은 차이도 업무 요건과 대조해 원인을 확인했습니다.**

문제 해결 과정

문제

Excel로 수작업 관리하던 원가 산정 업무를 시스템으로 옮기면서, 개발 단계·품목·원가요소 간 관계와 다양한 계산 조건을 반영해야 했습니다.

내가 한 일

구매원가팀과 요구사항을 검증하며 환율·관리비·LDC율·단가에 따른 원가 차이 산정 로직을 설계했습니다. 현업 계산식과 시스템 결과를 비교하고, 환차액 누락으로 최종 단가에 0.07% 차이가 발생하는 원인을 분석했습니다.

결과

누락된 계산 요인을 확인해 결과의 정합성을 확보했습니다. 반복 계산을 시스템화하고, 원가 차액을 원인별로 살펴볼 수 있도록 결과를 제공했습니다.

PRODUCTION EXPERIENCE / 02

REQUEST ↔ ISSUE

문의 · 진행 이력 · 첨부파일 연동

데이터 구조 개선 · API 개발 2026.01 — 2026.05

고객문의 구조 개선과 Jira 연동

emroCloud · 데이터 모델 / 외부 연동

Java Spring Oracle Jira REST API AWS S3

> **문의 모델과 외부 연동 방식을 함께 바꿔 대화형 이력을 관리하도록 개선했습니다.**

문제 해결 과정

문제

기존의 1문의 1답변 구조로는 대화형 문의 이력을 관리하기 어려웠습니다. Jira 연동에서는 첨부파일과 콘텐츠 제약사항을 함께 처리해야 했습니다.

내가 한 일

요청과 진행 이력을 분리한 정규화 구조로 개선하고, 상태별 데이터 이관 기준을 설계했습니다. 문의·진행 내용·첨부파일을 Jira Issue와 Comment에 연결하는 REST API를 개발하고 Multipart 전송 및 파일명·댓글 링크 제약을 분석했습니다.

결과

기존 데이터를 대화형 구조로 이관했습니다. Base64 이미지로 인한 콘텐츠 제한 문제는 S3 기반 구조로 변경해 연동 안정성을 개선했습니다.

PRODUCTION EXPERIENCE / 03

STATE → PROCESS

중복 요청 방지와 오류 로그 보존

동시성 제어 · 예외 처리 개선 2025.09 — 2025.11

일괄결재 처리 안정성 개선

emroCloud · 동시성 / 트랜잭션

Java Spring Oracle MyBatis

> **시작 상태 기록과 건별 결재의 트랜잭션 경계를 분리했습니다.**

문제 해결 과정

상태 변경과 실제 결재 처리의 시점이 어긋난 구조를, 처리중 상태와 건별 트랜잭션으로 나누어 개선했습니다.

AS-IS · 실제 처리 전에 완료 상태로 변경

선택한 결재를 먼저 '결재완료'로 갱신한 다음 각 건의 결재 전처리를 실행했습니다. 전체 작업의 트랜잭션을 분리하지 않아, 사용자가 다시 일괄결재를 요청했을 때 같은 결재 처리가 중복 실행되면서 데이터 정합성 오류가 발생했습니다.

```
-- AS-IS: 상태를 먼저 완료로 변경
UPDATE APPROVAL
   SET APPROVAL_STATUS = 'COMPLETED'
 WHERE APPROVAL_ID IN (/* 선택한 결재 ID */);

-- 이후 각 건의 결재 전처리 수행
-- 전체 작업의 트랜잭션 경계를 분리하지 않음
```

문제의 핵심 · 상태와 처리 단계의 불일치

'완료로 표시하는 시점'과 '실제 결재가 끝나는 시점'이 달랐습니다. 시작 상태를 기록하는 작업과 개별 결재를 처리하는 작업의 경계도 구분할 필요가 있었습니다.

개선에서는 '일괄결재처리중' 상태를 명시하고, 실제 건별 처리를 별도 신규 트랜잭션으로 실행하도록 나눴습니다.

TO-BE · 시작 상태 기록 → 건별 처리 → 조건부 완료

---

1. **시작 상태를 기록하는 트랜잭션**

   일괄결재 시작 시 이미 '일괄결재처리중'이거나 '결재완료'인 건은 제외하고, 나머지 대상의 상태를 '일괄결재처리중'으로 변경합니다. 재요청이 처리중·완료 상태를 다시 덮어쓰지 않도록 했습니다.

   ```
   -- 트랜잭션 1: 처리중 · 완료 상태는 대상에서 제외
   UPDATE APPROVAL
      SET APPROVAL_STATUS = 'BATCH_PROCESSING'
    WHERE APPROVAL_ID IN (/* 선택한 결재 ID */)
      AND APPROVAL_STATUS NOT IN
          ('BATCH_PROCESSING', 'COMPLETED');
   ```

2. **결재 건마다 신규 트랜잭션으로 처리**

   건별 전처리 모듈을 실행한 후, 해당 결재 ID의 현재 상태가 '일괄결재처리중'인 경우에만 완료 상태로 갱신합니다. 시작 상태 변경과 개별 결재 처리의 트랜잭션 경계를 분리했습니다.

```
-- 트랜잭션 2..N: 건별 신규 트랜잭션
-- requirednew = true
-- 1. 해당 결재 건의 전처리 모듈 실행
-- 2. 처리중 상태인 경우에만 완료로 변경
UPDATE APPROVAL
   SET APPROVAL_STATUS = 'COMPLETED'
 WHERE APPROVAL_ID = :approval_id
   AND APPROVAL_STATUS = 'BATCH_PROCESSING';
```

제공된 구현 흐름을 설명하기 위한 축약 SQL입니다. 테이블·컬럼·상태값은 설명용 이름이며, 대상 선별과 예외 처리의 전체 구현을 뜻하지 않습니다.

오류 로그는 업무 롤백과 분리

업무 처리 중 예외가 나더라도 원인을 확인할 수 있도록 비즈니스 트랜잭션과 오류 로그 트랜잭션을 분리했습니다. 업무 실패가 오류 로그 기록까지 되돌리지 않도록 구조를 개선했습니다.

개선한 점

처리중·완료 건을 재요청 대상에서 제외하고, 일괄결재의 시작과 완료를 서로 다른 상태로 표현했습니다. 실제 처리 단위는 건별 신규 트랜잭션으로 분리했습니다. 완료 갱신에는 현재 상태 조건을 추가해 의도한 단계의 결재만 완료로 전환하도록 했습니다.

PRODUCTION EXPERIENCE / 04

약 2억 건

운영 부하를 고려한 데이터 정리 대상

데이터 분석 · Batch Delete

2025.07 — 2025.10

대규모 운영 데이터 정리

emroCloud · 로그 / 메일 발송 이력

Oracle SQL ROWID Batch Delete

> **Archive Log 증가를 확인하고, ROWID 기반 삭제의 배치 건수를 더 작게 제한했습니다.**

문제 해결 과정

보존기간 조건만으로 대량 DELETE를 실행하던 방식에서, Archive Log 증가를 확인하고 삭제 건수와 트랜잭션 크기를 명시적으로 제한하는 방식으로 바꿨습니다.

AS-IS · 기간 조건에 해당하는 데이터를 일괄 삭제

로그와 메일 발송 이력 등 보존기간이 지난 데이터를 정리하는 작업이었습니다. 약 2억 건 규모의 정리 대상을 분석했으며, 날짜 조건만 있는 DELETE는 한 번의 작업이 처리할 건수를 명확히 제한하지 못했습니다.

```
-- AS-IS: 보존기간이 지난 전체 데이터를 한 번에 삭제
DELETE FROM LOG_TABLE
 WHERE CREATE_DATE < SYSDATE - :retention_days;
```

운영 중 확인한 문제 · 삭제에도 로그 공간이 필요

데이터를 삭제하는 작업에서도 변경 내용에 대한 Redo가 발생하고, Archive Log가 쌓여 용량이 급증하는 점을 확인했습니다. 삭제 대상의 용량뿐 아니라 작업이 새로 발생시키는 로그 공간도 고려해야 했습니다.

한 번의 처리 분량을 더 작게 조정하고, 스크립트에서 처리 건수를 명확히 제한하는 방향으로 변경했습니다.

TO-BE · ROWID 커서와 건수 제한으로 작업 분할

1. **보존기간을 지난 대상의 ROWID 조회**

   날짜 조건은 유지하고, 커서가 삭제할 행의 ROWID를 가져오도록 구성했습니다.

2. **일정 건수만 FETCH하고 해당 ROWID 삭제**

   한 번에 가져오는 건수를 제한하고, 이번 묶음의 ROWID만 DELETE 대상으로 사용했습니다. 전체 대상 건수와 한 번의 처리 건수를 구분했습니다.

3. **일정 건수마다 COMMIT 후 다음 묶음 처리**

   대상 전체를 하나의 큰 트랜잭션으로 처리하지 않고 작업을 나눴습니다. Archive Log 증가를 확인한 뒤 배치 분량을 더 타이트하게 줄여 처리 단위를 조정했습니다.

```
-- TO-BE: ROWID 조회 후 제한된 건수씩 처리
CURSOR c_log IS
    SELECT ROWID
      FROM LOG_TABLE
     WHERE CREATE_DATE < SYSDATE - retention_days;

-- 아래는 반복 처리 흐름을 나타낸 의사 코드
FETCH c_log BULK COLLECT INTO l_rowids
      LIMIT batch_size;

DELETE FROM LOG_TABLE
 WHERE ROWID IN (/* 이번 FETCH에서 가져온 ROWID */);

COMMIT;
-- 다음 묶음을 FETCH하여 반복
```

커서·FETCH·DELETE·COMMIT의 구조를 보여주는 의사 코드입니다. 실제 배치 크기와 삭제 구현의 세부 문법은 생략했습니다.

개선의 의미와 한계

배치별 삭제 건수와 트랜잭션 크기를 제어할 수 있도록 바꿨습니다. 작은 배치와 COMMIT이 같은 데이터를 삭제할 때 발생하는 전체 Redo·Archive Log 양을 없애거나 자동으로 줄여 주는 것은 아닙니다. 작업 단위를 줄이는 것과 누적 로그 공간을 확보하는 것은 함께 고려해야 할 문제입니다.

> **확장 시 고려할 대안 · 날짜 기준 파티션**
>
> 날짜 기준으로 파티션을 구성할 수 있고 보존기간 경계와 파티션 경계가 맞는 환경이라면, 오래된 파티션을 DROP하는 방식도 검토할 수 있습니다. 행별 DELETE를 반복하는 대신 보존기간 관리를 파티션 단위 작업으로 바꾸는 접근입니다. 이번 운영 정리에 실제 적용한 방식은 ROWID 기반 분할 DELETE입니다.
>
> ```
> -- 대안 예시: 날짜 기준 파티션 구성이 전제
> ALTER TABLE LOG_TABLE
>     DROP PARTITION P_2021;
> ```
>
> 실제 적용에는 기존 테이블 전환과 인덱스·참조 제약의 영향을 검토해야 합니다. Oracle 19c의 일반적인 온프레미스 EE 구성에서는 Oracle Partitioning이 별도 옵션이므로, 'Enterprise Edition이면 바로 사용 가능'으로 단정하지 않고 기능과 사용 권한을 함께 확인해야 합니다.
>
> Oracle 파티션 유지관리 문서 ↗ (https://docs.oracle.com/en/database/oracle/oracle-database/19/vldbg/maintenance-partition-tables-indexes.html)
> Oracle 19c 기능·옵션 안내 ↗ (https://docs.oracle.com/en/database/oracle/oracle-database/19/dblic/Licensing-Information.html)

---

03 / SKILLS

직접 사용하며 익힌 기술

기술 이름과 실제 사용 경험을 함께 확인하세요

LANGUAGE

Java

B2B 서비스의 업무 로직과 데이터 처리 개발

실무

원가 산정 로직 ↗

BACKEND

Spring

REST API · 트랜잭션 · 인증과 인가

실무

결재 트랜잭션 개선 ↗

DATABASE

Oracle

SQL 작성 · 대량 데이터 정리 · 데이터 구조 개선

실무

대량 데이터 정리 ↗

DATA ACCESS

MyBatis

업무 요건에 맞춘 SQL과 객체 매핑

실무

데이터 모델과 연동 SQL ↗



INFRASTRUCTURE

Docker

팀 프로젝트 컨테이너 배포와 네트워크 구성

팀 프로젝트

컨테이너 배포 구성 ↗



DATABASE

MySQL

취업 전 프로젝트 데이터 모델링과 저장

팀 프로젝트

프로젝트 데이터 모델링 ↗

CLOUD

AWS EC2 / S3

프로젝트 서버 운영 · S3 기반 파일 연동

실무 · 팀 프로젝트

S3 파일 연동 ↗



CI/CD

Jenkins

팀 프로젝트 빌드와 배포 파이프라인 구축

팀 프로젝트

빌드·배포 파이프라인 ↗

함께 사용한 기술

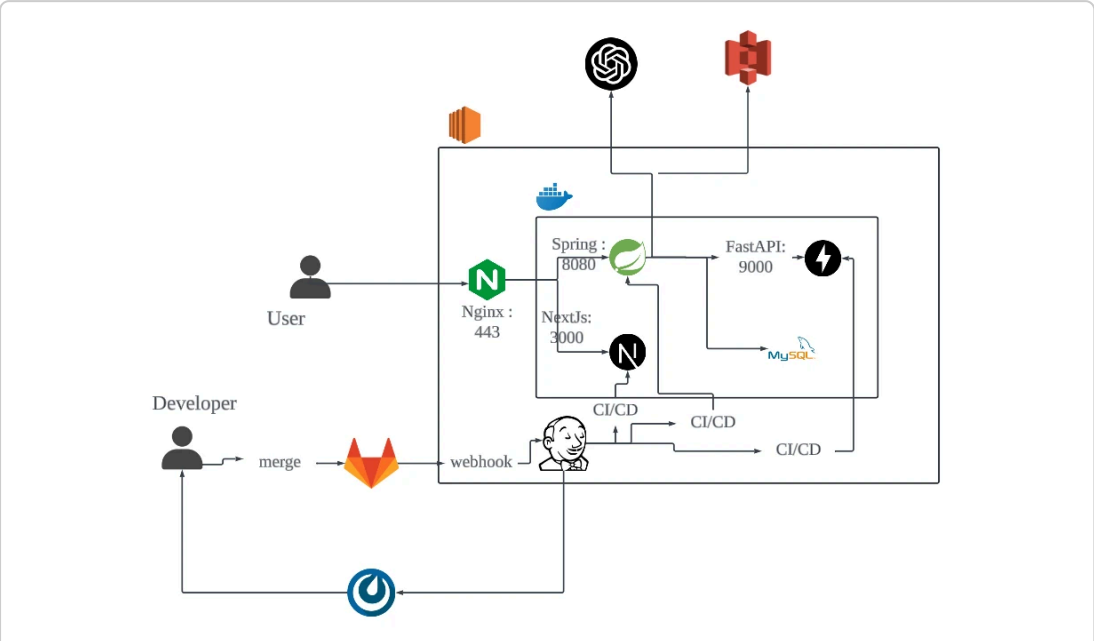
DB2 · JPA · Querydsl · Git · Jira

---

04 / TEAM PROJECTS · 2023 — 2024

직접 만들며 배운 기본기

취업 전 팀 프로젝트에서 백엔드 개발과 배포 환경을 함께 경험했습니다.



아키텍처

사용자 요청은 Nginx를 거쳐 Next.js 화면과 Spring API로 전달됩니다. 그림은 Spring·FastAPI·MySQL과 외부 API 및 S3의 연결, GitLab 머지에서 Jenkins로 이어지는 배포 경로를 함께 보여줍니다. 저는 뉴스 API와 EC2·Docker 운영 환경, Jenkins 파이프라인을 담당했습니다.

팀장 · 백엔드 · 인프라

2024.03 — 2024.04 · 6주 / 6인

여의도꿀통

국회의원 활동현황 · 정치뉴스 서비스

Spring Boot | Querydsl | EC2 | Docker | Jenkins

문제 해결 과정

문제

뉴스와 연관 키워드를 함께 조회하는 1:N 관계에서 페이징과 Fetch Join을 함께 적용하면서 메모리 페이징 경고가 발생했습니다.

내가 한 일

페이지에 해당하는 ID를 먼저 조회하고, 해당 ID의 연관 데이터를 Fetch Join으로 가져오도록 쿼리를 분리했습니다. 뉴스 API 개발, EC2·Docker 배포 환경 관리와 Jenkins CI/CD 구축도 담당했습니다.

결과

페이징 대상과 연관 데이터 조회 단계를 분리했습니다. Docker와 UFW의 필터 적용 순서도 점검해 의도한 방화벽 정책을 적용했습니다.

설계와 개선 과정



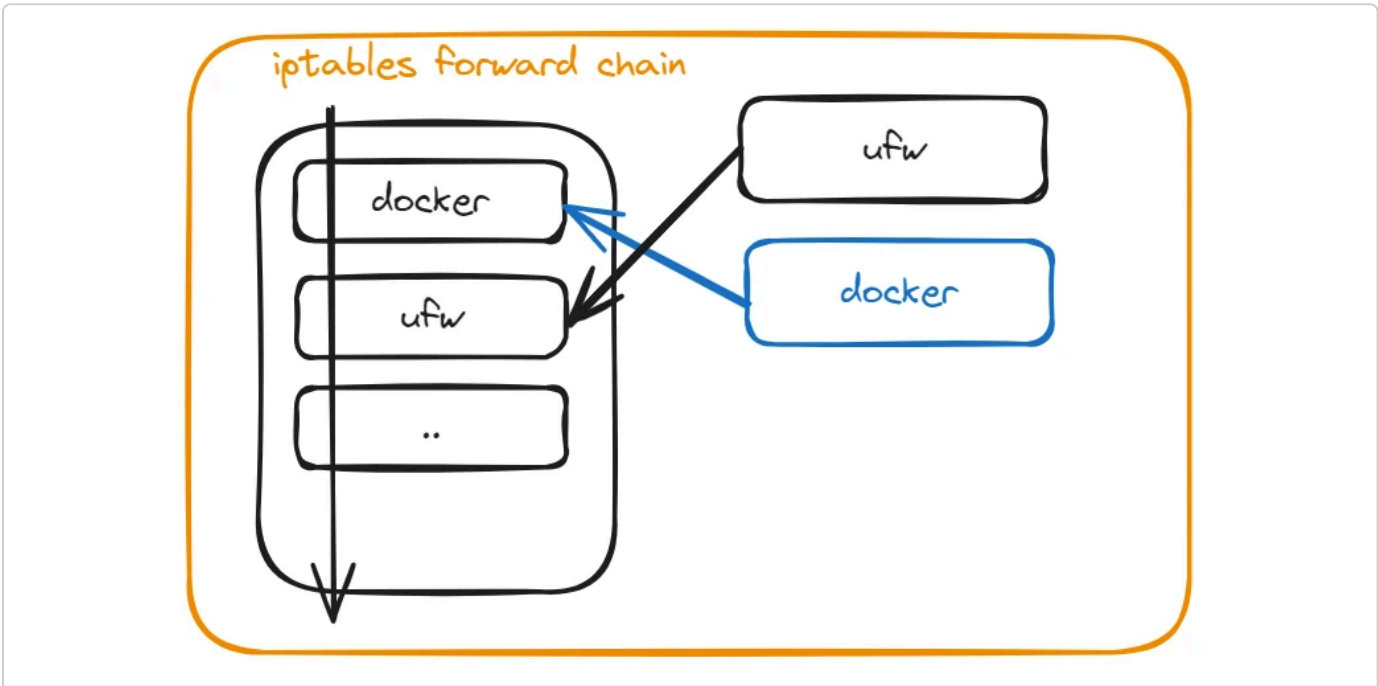
페이지네이션 문제 상황

변경 전에는 뉴스와 여러 키워드를 한 번에 Fetch Join하면서 페이지 제한도 적용했습니다. 뉴스 한 건이 키워드 수만큼 행으로 펼쳐져, 컬렉션 조인과 페이징을 함께 처리할 때 메모리 페이징 경고가 발생했습니다.



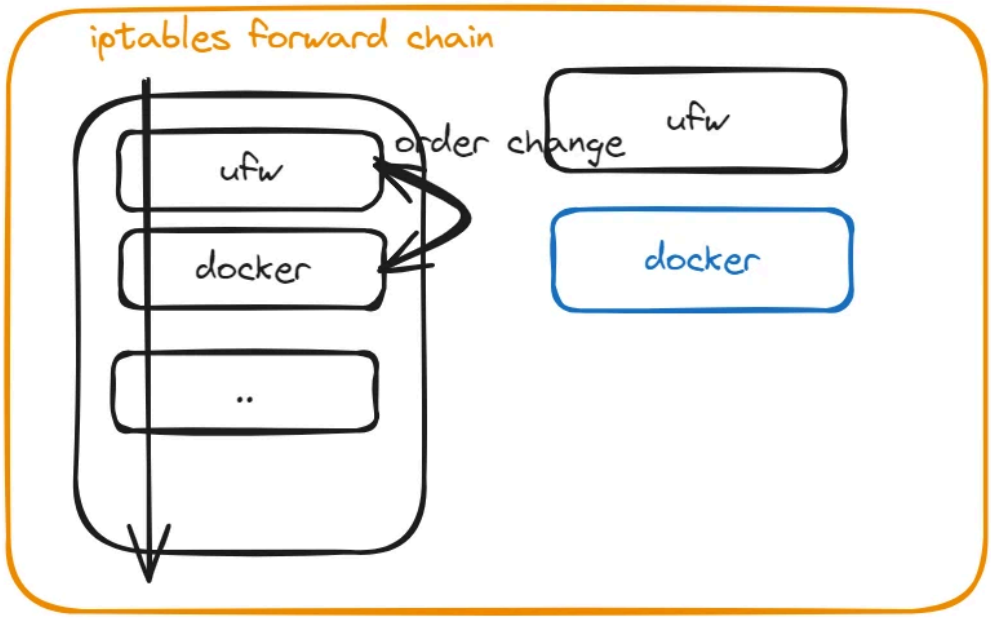
ID 조회와 연관 데이터 조회 분리

변경 후에는 정렬·페이지 조건에 맞는 뉴스 ID를 먼저 구하고, 그 ID 목록에 해당하는 뉴스와 키워드를 Fetch Join으로 조회했습니다. 페이지 경계를 결정하는 조회와 연관 데이터를 채우는 조회를 분리한 것이 핵심입니다.



방화벽 필터 변경 전

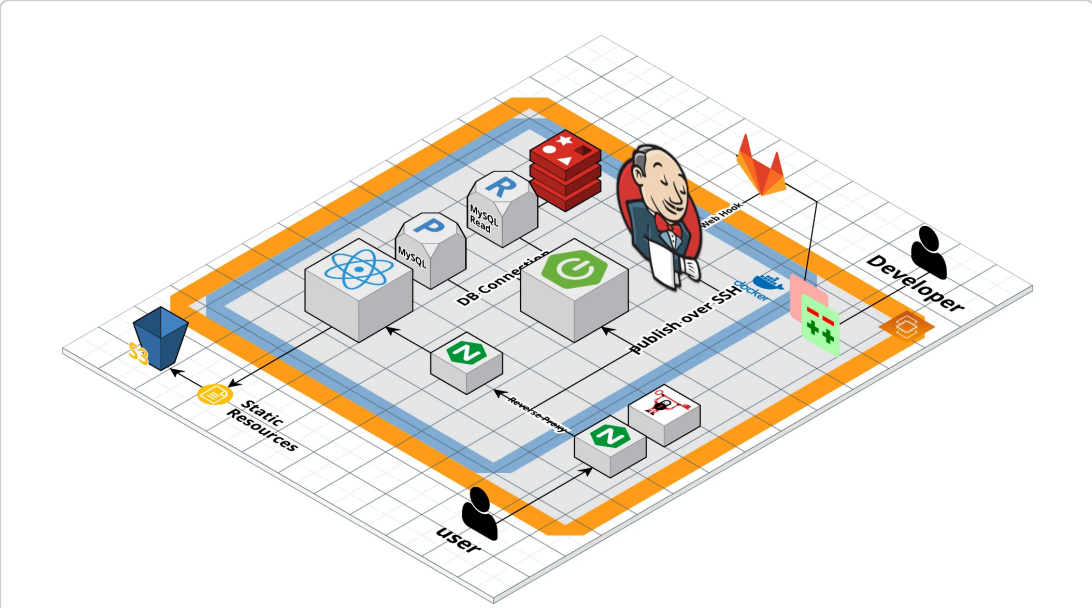
Docker로 포트를 공개한 뒤에는 UFW 설정만으로 의도한 접근 제한이 적용되지 않는 문제가 있었습니다. 그림은 컨테이너로 향하는 트래픽과 방화벽 필터의 적용 순서를 확인한 변경 전 상황입니다.



방화벽 필터 변경 후

Docker와 UFW의 필터 적용 순서를 점검해 컨테이너로 들어가는 경로에도 의도한 정책이 적용되도록 변경했습니다. 애플리케이션 설정뿐 아니라 실제 패킷이 지나는 경로를 확인해야 한다는 운영 경험을 얻었습니다.

GitHub 저장소 ↗ (https://github.com/JaegeonYu/HoneyPot)



아키텍처

사용자 트래픽은 외부 프록시에서 내부 Nginx와 React·Spring 서비스로 이어집니다. 데이터 계층에는 MySQL과 Redis, 정적 리소스 경로에는 S3가 표시되어 있습니다. 개발자의 GitLab 변경은 Jenkins와 SSH 배포 경로로 이어지며, 저는 백엔드 API·팀 단위 인가와 배포 자동화를 담당했습니다.

팀장 · 백엔드 · 인프라

2024.01 — 2024.02 · 7주 / 6인

시작

스토리 작가를 위한 작품 설정 시각화

Spring Boot | Spring Security | JUnit 5 | Docker | Jenkins

문제 해결 과정

문제

수작업 배포는 담당자에게 의존했고, 프론트엔드와 백엔드 연동 과정에서는 CORS 오류가 발생했습니다.

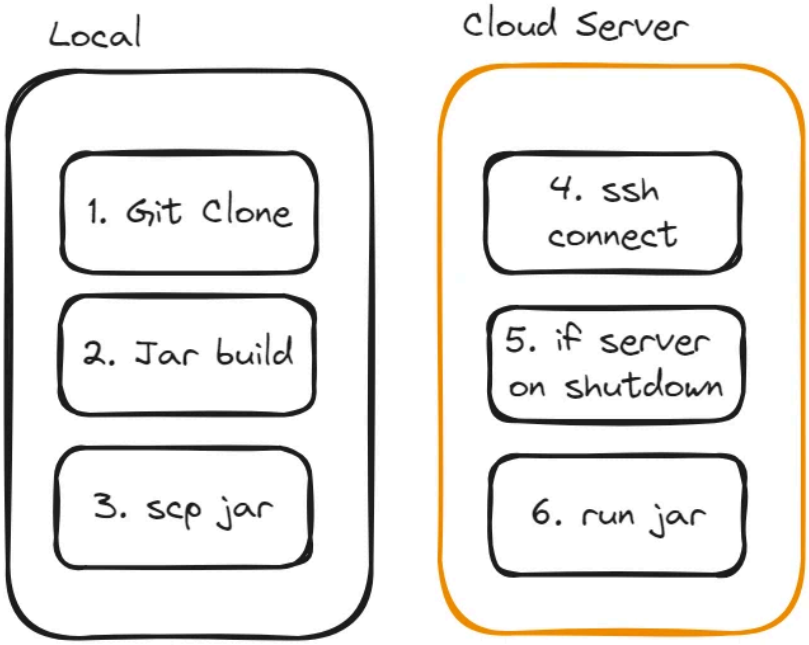
내가 한 일

팀·유저 API와 Spring Security Voter 기반 팀 단위 인가 로직, JUnit 5 테스트를 구현했습니다. Jenkins 파이프라인과 리버스 프록시를 구축하고, CORS에서는 Origin과 OPTIONS 요청 처리를 점검했습니다.

결과

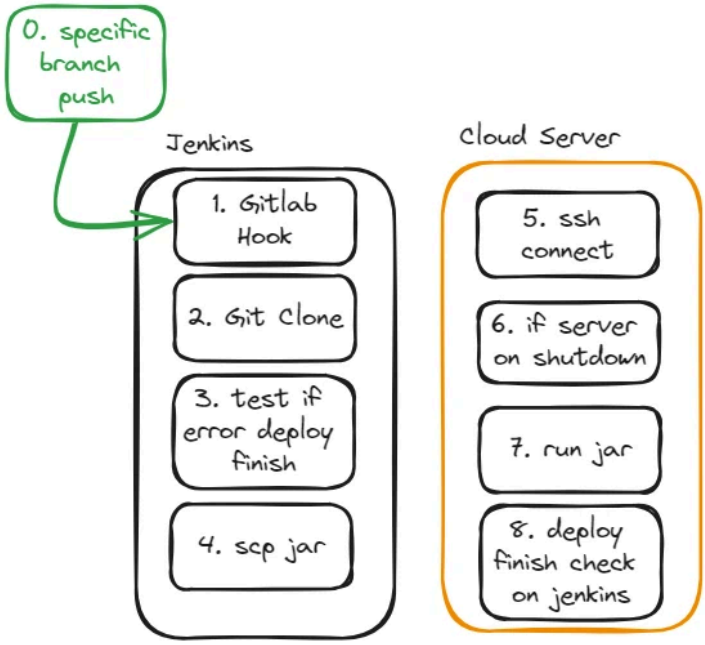
브랜치 머지를 통한 자동 배포 환경을 구축했습니다. 외부 접근을 프록시로 모으고, 프록시에서 SSL을 처리하도록 배포 구조를 구성했습니다.

설계와 개선 과정



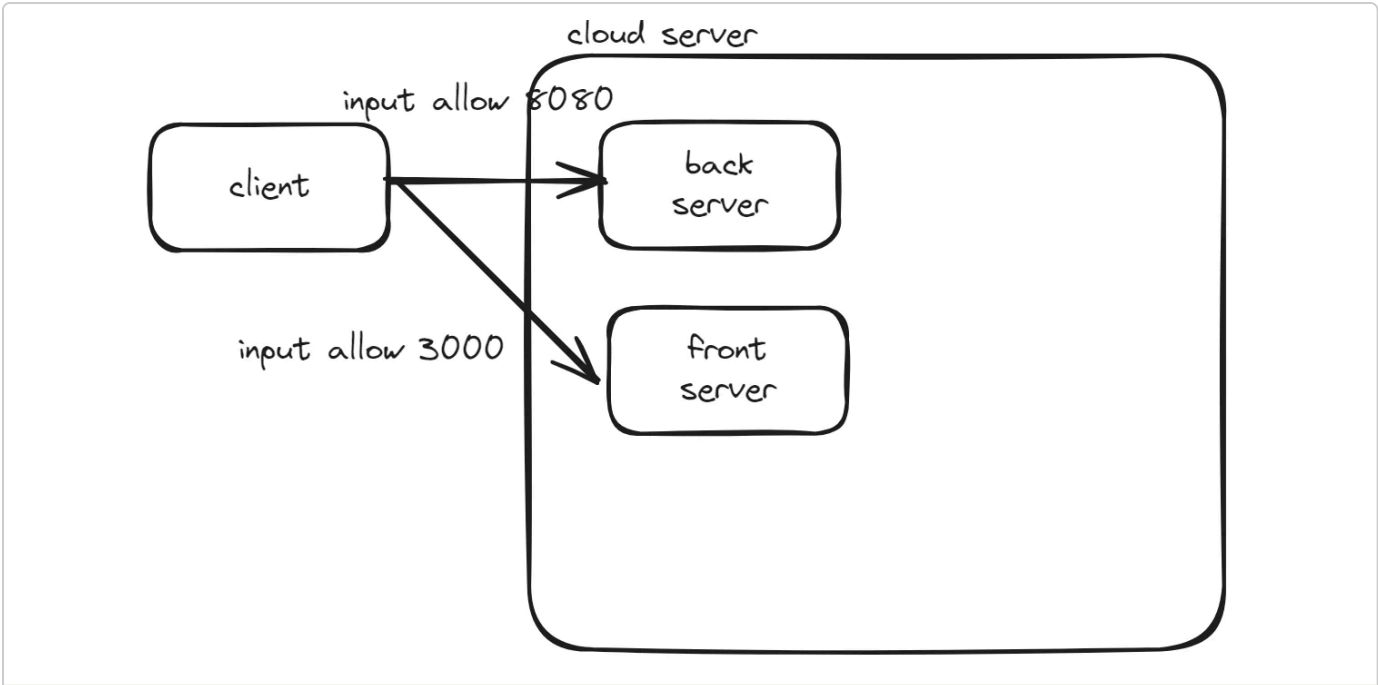
수동 배포 과정

변경 전 배포는 담당자가 직접 빌드와 서버 반영을 진행하는 흐름이었습니다. 반복 작업이 사람에게 의존해 팀원이 기능 개발과 배포 진행 상황을 따로 맞춰야 했습니다.



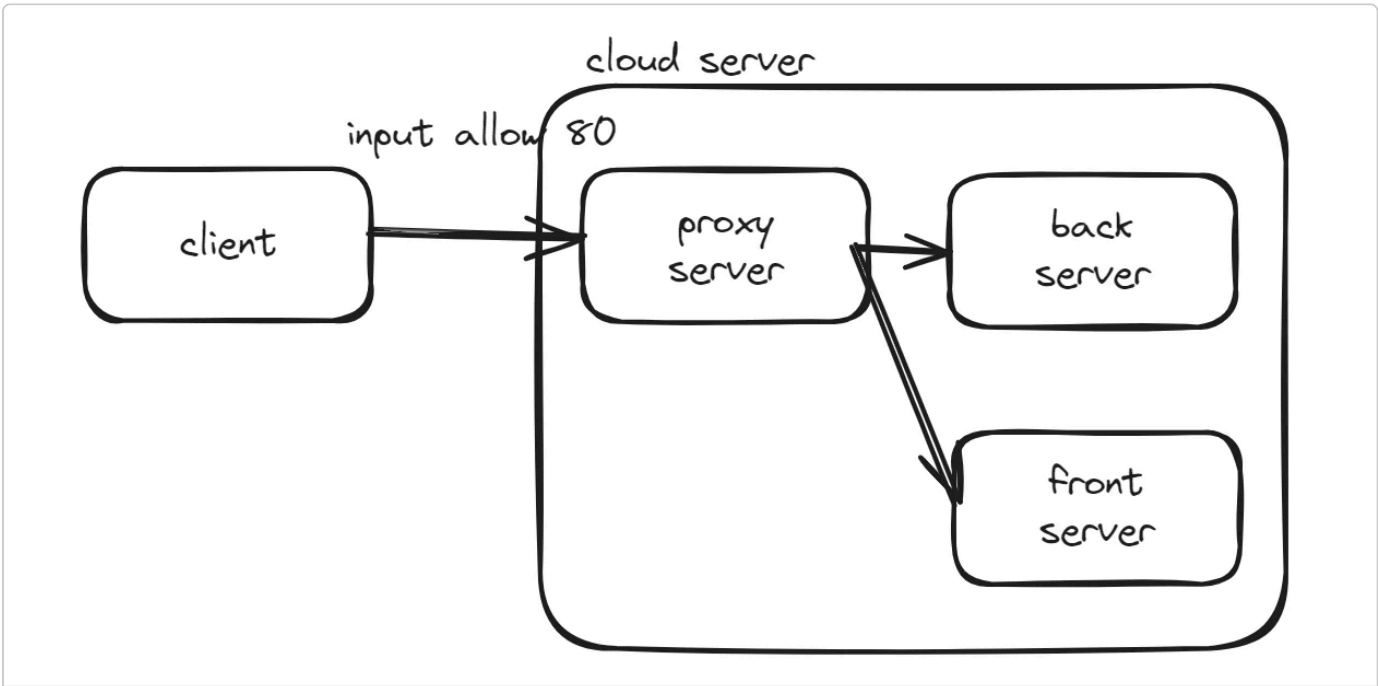
Jenkins 자동 배포 과정

변경 후에는 브랜치 머지가 Jenkins 파이프라인 실행으로 이어지도록 구성했습니다. 빌드와 배포 절차를 파이프라인으로 옮겨, 수작업으로 수행하던 단계를 공통된 실행 흐름으로 만들었습니다.



리버스 프록시 적용 전

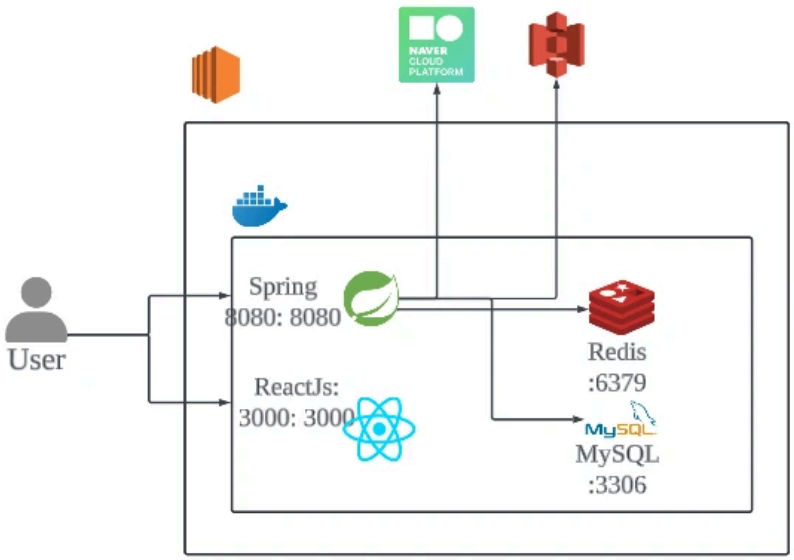
변경 전 구조와 비교하며 프론트엔드·백엔드의 외부 접근 경로를 점검했습니다. 연동 과정의 CORS 오류는 요청 Origin과 사전 요청인 OPTIONS가 어떻게 처리되는지 나누어 확인했습니다.



리버스 프록시 적용 후

변경 후에는 외부 접근을 리버스 프록시로 모으고 SSL 처리를 프록시에서 맡도록 구성했습니다. 외부 진입점과 내부 서비스 경로를 구분했으며, CORS 문제는 Origin·OPTIONS 요청 설정을 함께 점검해 해결했습니다.

GitHub 저장소 ↗ (https://github.com/JaegeonYu/PerspectiView)



아키텍처

EC2 안에서 Docker로 애플리케이션과 MySQL·Redis를 구성하고, Spring 서버가 데이터 저장소 및 외부 서비스와 통신하는 구조입니다. 저는 API·인증·알림과 배포 환경을 담당했습니다. 이 과거 아키텍처 이미지의 프론트엔드 표기는 ReactJs이며, 프로젝트 개요 자료의 Vue.js 표기와 차이가 있어 원본 그대로 남겼습니다.

백엔드 · 인프라 · 협업 설계

2023.11 · 2주 / 2인

Yesterday's Trip

관광지 리뷰 · 관심지역 실시간 알림

Spring Boot | MyBatis | JWT | SSE | EC2

문제 해결 과정

문제

이메일 인증 요청의 SMTP 통신 지연과 함께, API 명세서만으로는 프론트엔드 팀원과 처리 순서를 공유하기 어렵다는 문제가 있었습니다.

내가 한 일

멀티스레드를 활용해 SMTP 통신을 비동기 처리했습니다. Open API 데이터 적재, 관광지·유저·리뷰·지역 API, JWT 인증·인가, SSE 기반 관심지역 실시간 알림과 배포 환경도 담당했습니다.

결과

당시 프로젝트 측정에서 이메일 인증 요청 응답을 3초 이상에서 0.5초 미만으로 줄였습니다. 이 수치는 요청 응답 시간이며 이메일 도착 시간을 의미하지 않습니다.

시퀀스 다이어그램으로 협업

API 명세서만으로 설명했을 때 생기던 구현 방향의 차이를 줄이기 위해, 지역 팔로우·회원가입·로그인의 요청과 응답 흐름을 시퀀스 다이어그램으로 정리해 GitHub에 공유했습니다. 프론트엔드 팀원과 처리 순서를 시각적으로 확인하며 더 빠르고 정확하게 의견을 맞추고 피드백을 받을 수 있었습니다.

실시간 알림 로직

1. 사용자가 관심 지역을 팔로우합니다.
2. 관리자가 해당 지역에 콘텐츠를 추가하면, 팔로워의 알림을 DB에 저장합니다.
3. 접속한 사용자에게는 SSE로 알림 ID를 전송하고, 클라이언트는 ID로 상세 내용을 조회합니다.
4. 미접속 사용자는 로그인 후 SSE 연결 시 읽지 않은 알림을 전달받습니다.

서버에서 클라이언트로 보내는 단방향 알림에 맞춰 SSE를 선택했습니다. 연결의 생성·종료 콜백을 고려해 Emitter 저장소에는 ConcurrentHashMap을 사용했습니다.

SQL 처리 방식 개선

팔로워를 조회한 뒤 반복해서 알림을 삽입하던 처리를 INSERT INTO … SELECT로 바꿔 다중행 삽입을 한 문장으로 처리했습니다. 선택한 알림의 읽음 처리는 MyBatis foreach와 IN 조건으로 묶었습니다.

SSE를 선택한 이유와 연결 관리

알림은 서버에서 사용자에게 보내는 단방향 통신이어서 SSE를 선택했습니다. 주기적으로 변경을 확인하는 Polling 대신 연결을 유지하고, Spring의 SseEmitter를 사용해 짧은 프로젝트 일정 안에 구현했습니다. 사용자 ID별 연결은 ConcurrentHashMap에 보관하고 종료·시간 초과 콜백에서 정리했습니다.

구현 범위와 남은 과제

연결 저장소는 단일 서버의 메모리에 있습니다. 서버를 여러 대로 늘리면 다른 서버의 접속자에게도 이벤트를 전달할 수 있도록 메시지 전달 계층이 필요합니다. Redis Pub/Sub이나 Kafka 연계는 확장 시 검토할 과제이며, 당시 구현에 포함된 기능은 아닙니다.

MyBatis로 관계와 조회 범위 표현

사용자와 여러 권한의 관계는 결과 매핑으로 표현했습니다. 알림 생성은 팔로워 조회 후 건별 INSERT하던 1+N회 호출을 INSERT INTO … SELECT 한 문장으로 합쳤고, 선택한 알림의 읽음 처리는 foreach로 IN 조건을 구성했습니다.

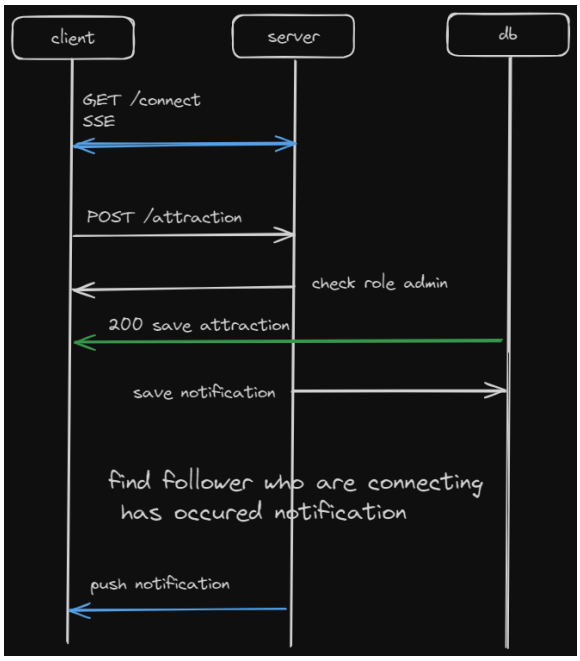
관광지 추천에서는 전체 결과를 애플리케이션에서 잘라 쓰는 대신, 리뷰 수를 구하는 상관 서브쿼리와 평점 정렬·LIMIT을 같은 SQL 안에서 처리했습니다. 필요한 결과의 범위를 DB 조회 단계에서 결정하도록 바꾼 경험입니다.

GitHub에 공유한 시퀀스와 로직



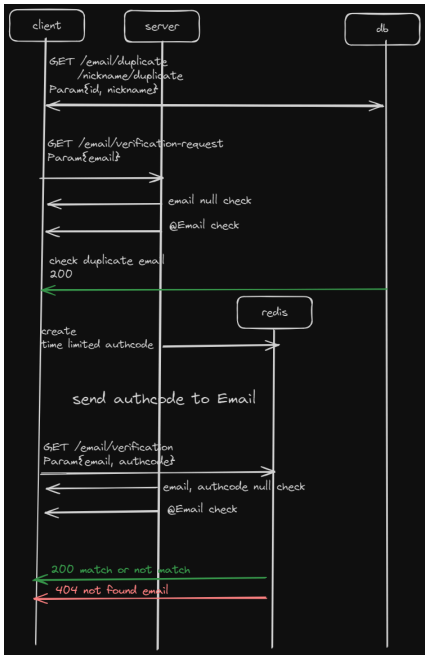
지역 팔로우 시퀀스

클라이언트의 지역 팔로우 요청이 서버와 DB로 이어지는 순서를 공유한 그림입니다. 관심 지역을 저장하는 단계가 이후 알림 수신 대상을 결정하는 기준이 되도록 프론트엔드 팀원과 요청·응답 순서를 맞췄습니다.



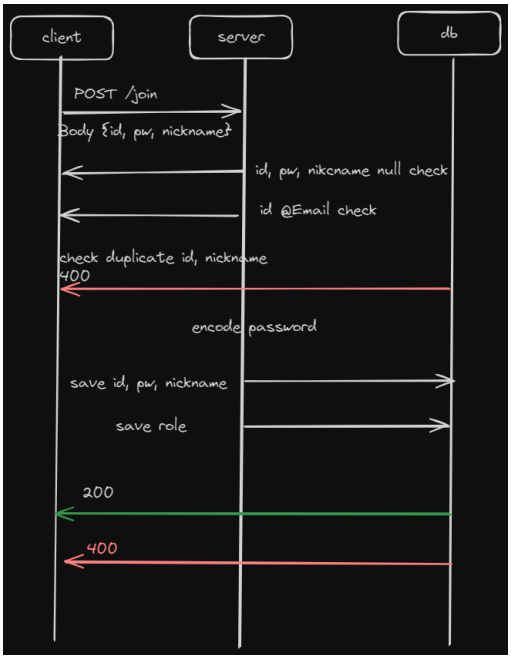
실시간 알림 시퀀스

지역 관리자의 콘텐츠 등록 이후 팔로워에게 알림이 전달되는 순서를 정리했습니다. 알림을 DB에 남기는 단계와 현재 접속한 사용자에게 SSE 이벤트를 보내는 단계를 구분해, 화면에서 상세 내용을 조회하는 시점을 공유했습니다.



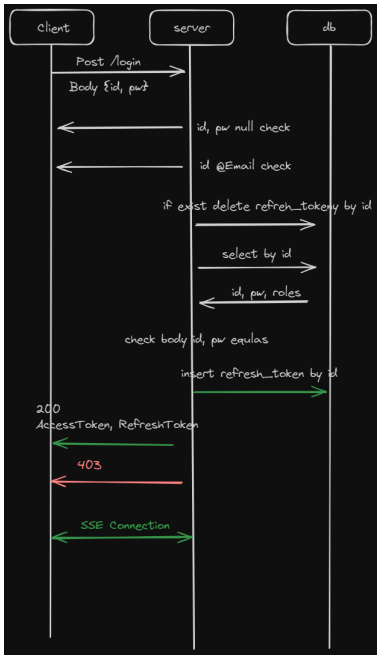
회원가입 전 이메일 인증 흐름

이메일·닉네임 중복 확인 후 인증 요청을 받으면 입력값을 검증하고, 유효기간이 있는 인증 코드를 Redis에 저장해 이메일로 보냅니다. 클라이언트가 이메일과 코드를 보내면 저장된 코드와 비교합니다. 그림의 화살표는 이 요청과 검증 결과의 반환 순서를 뜻합니다.



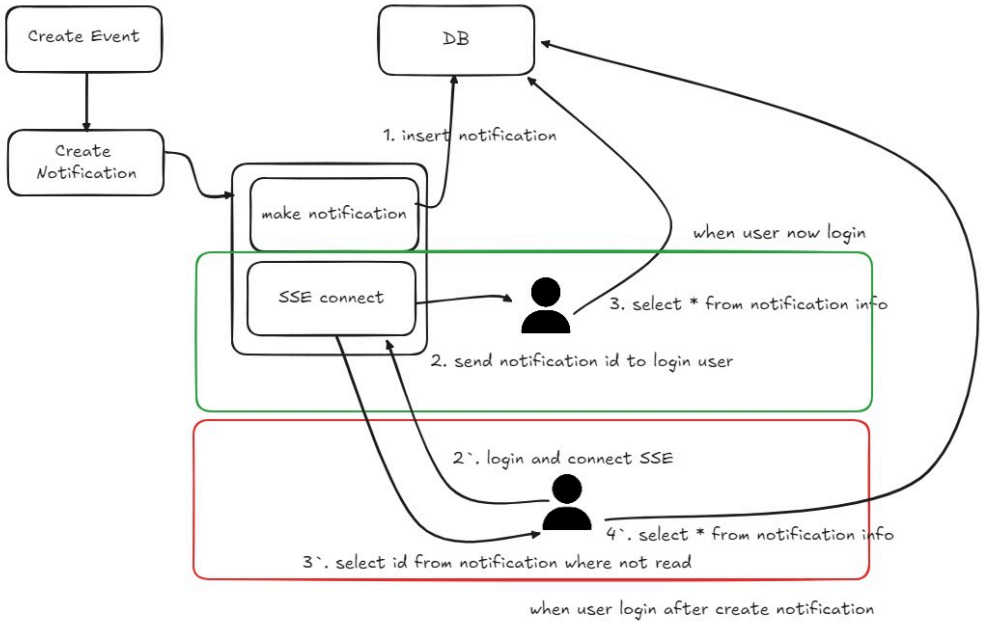
회원가입 처리 흐름

회원가입 요청의 필수값·이메일 형식·중복 여부를 검증한 뒤 비밀번호를 인코딩하고 계정과 권한을 저장합니다. 성공과 실패의 응답 흐름을 함께 표시해 프론트엔드가 어떤 결과를 처리해야 하는지 공유했습니다.



로그인 시퀀스

로그인 입력값을 확인하고 계정·권한을 조회해 인증한 뒤 Access Token과 Refresh Token을 반환하는 흐름입니다. 그림에는 기존 Refresh Token 정리와 새 토큰 저장, 로그인 이후 SSE 연결까지 표시해 인증 이후의 화면 처리 순서도 함께 맞췄습니다.



SSE 알림 저장과 전달 로직

관심 지역 콘텐츠가 등록되면 팔로워의 알림을 저장하고, 연결된 사용자에게 알림 ID를 보냅니다. 접속하지 않은 사용자의 알림은 DB에 남아 다음 로그인 시 미읽음 알림으로 전달됩니다. 저장과 실시간 전달의 역할을 나누어 설명한 그림입니다.

본문에 반영한 기술 기록 · 원문

MyBatis 쿼리 개선
다중행 삽입 · 동적 SQL · 연관 데이터 매핑 ↗
(https://anythingis.tistory.com/167)

SSE 실시간 알림
기술 선택 · 연결 관리 · 구현의 한계 ↗
(https://anythingis.tistory.com/168)

GitHub 저장소 ↗ (https://github.com/JaegeonYu/yesterday-s-Trip)

05 / HOW I WORK

요구사항부터 운영까지 맥락을 이어갑니다.

요구사항의 배경을 묻고, 데이터의 흐름을 살핍니다.

현업 담당자와 소통하며 업무의 맥락을 파악하고, 기능과 데이터의 정합성을 함께 고려해 개발합니다. 온프레미스 레거시 환경에서도 데이터 접근 경로와 I/O를 살피며 SQL과 처리 방식을 검토해 왔습니다.

실무에서 마주한 고민을 체계화하기 위해 데이터 모델링과 SQL 튜닝을 학습하며 SQLP를 취득했습니다. 주어진 시스템에서 개선할 지점을 꾸준히 찾아가는 개발자를 지향합니다.

신규 테이블의 인덱스는 선택도와 주요 조회·조인 패턴을 살펴 설계를 제안했습니다. 성능 문제를 운영 이후에만 다루기보다, 설계할 때부터 데이터 접근 방식을 함께 검토합니다.

NAME
유재건 · Jaegeon Yu

ROLE
Backend Developer

CURRENTLY
엠로(emro) · 2024.09 — 현재

FOCUS
데이터 정합성 · SQL · 서비스 운영

06 / LEARNING

경험을 지탱하는 학습

SQLP

2026.09

데이터 모델링과 SQL 튜닝 학습을 통해 실무의 고민을 체계화했습니다.

SQLD · 정보처리기사

2024.09 / 2023.06

데이터와 소프트웨어 개발의 기본기를 쌓았습니다.

삼성청년SW아카데미 10기

2023.07 시작 · 1년 수료

Java 기반 백엔드 교육과 팀 프로젝트를 수행했습니다. SSAFY 프로젝트 경진대회 우수상·최우수상을 수상했습니다.

한밭대학교 컴퓨터공학과

2017.03 — 2023.02

자료구조, 운영체제, 데이터베이스, 네트워크를 학습하고 지문 기반 본인인증 앱의 백엔드 개발을 담당했습니다.

---

07 / CONTACT

다음 경험을
함께 만들어가고 싶습니다.

업무의 맥락과 데이터의 흐름을 함께 고민하는 팀과
더 나은 서비스를 만들어가고 싶습니다.

yjk98053@gmail.com ↗

GitHub ↗ / 기술 블로그 ↗
